Maestro — customer-service agent for Empório da Música

Project documentation: README, specification, tool configuration and example conversations

Artefact AI Engineer technical challenge
Repository: https://github.com/barbieriht/emporio-da-musica-agent
Built from commit `662bca4`

Contents

1. Project README
2. Specification (SPEC.md)
3. Development-tool configuration (docs/agents)
4. Appendix: example conversations (Portuguese transcripts)

1. Project README

Source: `README.md`

Maestro — customer-service agent for Empório da Música

A Python prototype of a text-message customer-service agent for **Empório da Música**, a fictional musical-instrument store in Campo Grande, MS (Brazil), built for the Artefact AI Engineer technical challenge.

Maestro talks to customers in Brazilian Portuguese, in an informal but professional tone (as the store's manual asks). It looks up prices, stock, promotions and orders in the store's data, applies the manual's policies (opening hours, payment, shipping, exchanges, warranty), registers complaints and hand-off requests for the staff, and politely declines anything out of scope. Whatever has to be right (prices, deadlines, installments, order verification, opening hours) is tested code, not free text from the model.

- Example conversations (with the tools called in each turn): `examples/`
- Specification that guided the work: `SPEC.md` and issue #1
- How the project was built with Claude Code and the `mattpocock-skills` skills: section below
- All the documentation in a single PDF: `docs/maestro-documentation.pdf`

Language

Everything in the repository is written in **English** (README, specification, code, comments, test names, issues and commit messages), with one deliberate exception: **what the agent says to customers, and the text that instructs the model, is Brazilian Portuguese**. That covers the system prompt, the tool descriptions and tool outputs, the fixed messages (greeting, fallbacks, after-hours notice), the store's policy manual in `data/policies/` (the store's own source document) and the recorded example conversations. The customers are Brazilian and the data and manual are in Portuguese, so changing the agent's language would change the product, not just the documentation. The example files have English headers and Portuguese transcripts. For the same reason, the string literals inside the tests (what the agent is expected to say) are Portuguese.

How to run

Requirements: Python 3.12+ and uv.

```
git clone https://github.com/barbieriht/emporio-da-musica-agent.git
cd emporio-da-musica-agent
uv sync
cp .env.example .env        # fill in ONE provider key (see below)
uv run python -m emporio_agent
```

Type `sair` (Portuguese for "quit") to leave. To run the tests (they use neither the network nor a real LLM):

```
uv run pytest
```

Model provider

The agent uses LangChain's `init_chat_model`, so the provider is chosen with environment variables (`.env`):

Provider	Variables	Notes
Google Gemini (default)	`GOOGLE_API_KEY`	Free tier at https://aistudio.google.com/apikey. Default model pinned to `gemini-3.7-flash`.
Groq (alternative)	`GROQ_API_KEY`, `LLM_PROVIDER=groq`, `LLM_MODEL=openai/gpt-oss-120b`	Free tier with limits: https://console.groq.com/keys.

`LLM_PROVIDER` and `LLM_MODEL` override the default. **Model names change and get retired**: `gemini-2.5-flash` was already unavailable for new accounts during development, and a Groq Llama model that the assistant suggested from memory did not exist for the key. If you get `404 model not found`, list the models your key can use and set `LLM_MODEL`. If the provider answers `503 high demand`, the model is temporarily overloaded: retry, or switch model/provider.

Optional variables

Variable	Effect
`AGENT_TODAY=2026-02-20`	The sample orders are from 2025–2026, so with the real date every return would already be expired. This variable simulates the date **for the return rule only** (the "store open now" check and the after-hours notice always use the real clock).
`COMPLAINTS_FILE`	Where complaints and hand-off requests are written (default: `complaints.jsonl` in the project root, ignored by git).

Regenerating the examples

```
uv run python scripts/record_examples.py            # all of them
uv run python scripts/record_examples.py 04-real-order-return
```

The script replays scripted customer messages through the real agent and saves the transcripts with the tool calls. The output varies a little on every run (it is an LLM).

Regenerating the documentation PDF

```
uv run python scripts/build_docs_pdf.py
```

It renders this README, `SPEC.md`, `docs/agents/*.md` and the five example conversations into `docs/maestro-documentation.pdf`. It needs the `markdown` package (dev dependency, installed by `uv sync`) and the `wkhtmltopdf` program on the system.

How the agent works

```
customer message
      │
      ▼
 Agent.respond(session, message)
      │  ├─ tool loop (at most 5 model calls per message)
      │  ├─ checker: R$ values and dates in the final reply must come from the tools
      │  └─ after-hours notice (only on the first reply of a session in which the store is closed)
      ▼
 LLM (Maestro system prompt + session history + tool definitions)
      │ decides which tool to call
      ▼
┌──────────┬──────────┬────────┬─────────┬─────────────┐
│ Catalog │ Policies │ Orders │ Returns │ Complaints │
└──────────┴──────────┴────────┴─────────┴─────────────┘
  CSVs      manual as   CSVs +   pure       local
            .md files   verific. rules      file
```

The 11 tools exposed to the model:

Tool	What it does
`search_products`, `get_product_details`	Catalog search; details (description, specs, alternatives when unavailable)
`get_payment_options`	By product id: PIX (5% only without a promotion), debit, boleto, installments, combining payment methods
`get_policy`	A section of the manual (store, hours, payment, exchanges, shipping, promotions, service guidelines, warranty, privacy)
`get_installments`, `get_shipping_quote`, `is_store_open_now`	Installments for an amount, shipping by city, whether the store is open now
`get_order_status`, `check_return_eligibility`	Order status and return/exchange eligibility, with phone verification
`register_complaint`, `request_human_handoff`	Writes a complaint or a hand-off request, with a protocol number

Module	Responsibility
`agent.py`	`Agent.respond(session, message)`: tool loop, history, call limit, rollback if the model fails, after-hours notice, reply

	checking, tool access to the session
`grounding.py`	Extracts R$ values and dates from a text and checks them against what the tools/customer said
`prompt.py`	Persona (Maestro), conduct rules and the opening greeting
`catalog.py`	Loads products/categories/promotions; search, effective price, availability, alternatives
`policies.py`	Manual sections; installments, payment options, shipping, "store open now" and the after-hours notice (America/Campo_Grande time zone)
`orders.py`	Order lookup: format validation, phone verification and session memory of the verification
`returns.py`	Return/exchange eligibility by reason, deadline and order status
`complaints.py`	Local JSON Lines log of complaints and hand-offs, with a protocol number
`money.py`	Commercial rounding (half up) with `Decimal`
`tools.py`	Thin layer exposing the modules as LangChain *tools*, formatting answers in Portuguese
`config.py`, `cli.py`	Provider selection from env, clock (`AGENT_TODAY`), agent assembly, chat CLI

Technical decisions and rationale

Approach: function calling with deterministic tools (no vector RAG, no SQL agent). The data is small (65 products, 19 orders, 49 customers) and the manual is ~8 pages. Rules that must be right (promotional price, 7-day window, installments, shipping, opening hours) are testable code, not text for the model to interpret. The LLM keeps what it does well: understanding the question, choosing the tool and writing the answer in the store's tone. A SQL agent would write free-form queries over data with hidden business rules (active promotion, status beating stock); typed tools avoid that and give a small surface to test.

What is code and what is LLM (revised after a first version). In a second round I mapped what the model was still doing "in its head" and moved to code whatever could be deterministic: the **after-hours notice** (the manual requires it and before it only appeared if the customer asked), the **PIX discount** (the model used to compute it), the **registration of complaints** (the prompt said "register", but there was no tool: the agent claimed an action it did not perform), the **validation of phone and order number**, and the **checking of values and dates in the final reply**. Deliberately left to the LLM: understanding free language, classifying the return reason and writing the reply. I ruled out detecting "accessory" requests by keyword (it collides with "Cordas Orquestrais", a category the store does sell) and classifying intent with regexes.

Framework: LangChain (chat models and tools), with my own agent loop. I chose LangChain/LangGraph, but in the implementation only LangChain turned out to be needed: the loop (call model → run tools → return result, plus the call limit and the checker) fits in one short function, and writing it by hand kept the test seam simple (just inject a fake model). LangChain remains useful for switching providers by environment variable and for defining the tools. (I declared `langgraph` at the start and removed it later because it was unused.)

Model and provider: Gemini (default) and Groq (alternative), both free. Whoever evaluates this must be able to run it without paying; both have a free tier and support tool calling. The model is pinned by version (reproducibility) and can be switched by env. Local models (Ollama) were ruled out as the default: the agent depends on calling tools correctly, where small models fail more, and they require the evaluator's hardware.

Prompt strategy. The system prompt defines the persona and the manual's rules: never state a price, stock or deadline without consulting a tool; use the payment tool instead of computing PIX/installments; show the original price next to the promotional one; redirect accessory requests (the store only sells instruments); decline subjects unrelated to the store; do not make reservations or orders (the agent only informs; to buy, reserve or cancel it uses the hand-off); ask for phone and order number only on the first lookup; only say a complaint was registered if the tool confirms it; do not invent process steps or product features. The opening greeting is a fixed text (it costs no model call and works even when the provider is unstable), and the prompt tells the model it has already introduced itself.

Policy retrieval: Markdown sections, lookup by topic. I converted the PDF once into nine files in `data/policies/` (store, hours, payment, exchanges, shipping, promotions, service guidelines, warranty, privacy). That way the project runs without a PDF-reading dependency, the sections are reviewable, and the agent receives only the relevant excerpt. With a larger manual I would move to embeddings; for 8 pages, the cost and the uncertainty are not justified. (Risk: the conversion is manual and may drift from the original PDF, which stays in the repository.)

Data handling. The CSVs are loaded into memory at start-up and never modified. Normalizations: numeric prices, `specs` (JSON) as a dictionary, accent- and case-insensitive search with simple plurals ("violão" finds "Violões"), effective price with the active promotion, availability derived from status + stock. **Money uses `Decimal` with commercial rounding (half up)**: while checking numbers I found that `799.90 × 0.95 = 759.905` came out as `759.90` because of floating-point imprecision, when the correct commercial result is `759.91`.

Checker for values and dates in the final reply. The LLM writes the text and may alter or invent a value. After every final reply, the code extracts the R$ values and the dates; each one must appear in the session's tool results or in the customer's own messages (formats such as `R$ 1.000,00`, `1000` and `1 000` are equivalent; dates match by day/month, and by year when both sides carry it). Derived values (sums, differences) are **rejected** on purpose: calculations live in the tools. If something does not match, the agent asks for one rewrite, quoting the offending value (in that round it may call a tool); if that fails too, it sends a safe message without values, with the store's phone and WhatsApp. The rejected reply and the correction notice do not stay in the history.

Privacy and security of customer data. An order is only shown with **order number + registered phone** (compared by digits only); a non-existent order, a wrong phone and another customer's phone all give the same generic answer, without revealing which piece failed. The e-mail is not used because one of them appears on two customers. Formats are validated in code (phone with area code + 8/9 digits, positive integer order number). After a successful verification the session remembers the phone: it expires after 10 minutes without use, with a 30-minute cap since the customer typed the phone, it only covers that customer's orders, and wrong attempts are never remembered (nor do they erase a valid verification). The rule lives in code (`Orders.lookup`), not in the prompt, so the model cannot skip it. Complaints and hand-offs store **only the description and the contact** (never the conversation), in `complaints.jsonl`, outside git.

Conversation history. In memory, per session (`Session`). Persisting across sessions adds nothing to a CLI prototype. If the model fails in the middle of a turn, the history goes back to its previous state (no message left hanging without a reply).

Interface: CLI. The focus is the agent; a terminal chat is enough to evaluate it.

Testing strategy (163 tests, no network). Tests only at agreed public boundaries, never on internals: (1) `Agent.respond` with a scripted fake model (33 tests: history, tool loop, call limit, persona, rollback, after-hours notice, session context for the tools, value checker); (2) the `Catalog` (26), `Policies` (51), `Orders` (31), `assess_return` (12) and `Complaints` (10) interfaces over the real CSVs or temporary files, with injectable clock and state. Expected values come from literals checked against the CSVs/manual, not recomputed by the same code. **Outside** the tests: the CLI, the thin *tool* layers and the behaviour of the real LLM (checked manually and in the examples). Two notes on how much to trust the tests: I ran a mutation check on the checker (with it disabled, 5 tests fail), and the run with the real model found a *crash* that the unit tests missed (see "Assistant mistakes").

Assumptions

The brief asks to assume reasonable interpretations and document them. The main ones:

Catalog

- Only promotions with `is_active=1` count; if several are active on the same product, the largest wins (does not happen today).
- **Status beats stock**: product 136 (discontinued) shows stock 13 and 130 (coming soon) shows stock 2; both are treated as unavailable.
- Search by words (all must appear) in name, category and specifications; no typo tolerance. Up to 10 results, ordered by relevance → available first → discounted first → lower price. Unavailable products appear flagged.
- Price filters use the effective price (promotion already applied).
- Alternatives for an unavailable product: same category, available, price ±30%, closest first (up to 3).
- I do not sort by proximity: there is a single store and the products have no location.

Policies and payment

- Contacts: store phone (67) 3341-4444 and WhatsApp (67) 3321-4500 (the manual gives both, in different sections). E-mail `contato@emporiodamusica.com.br`; the table in §1.2 has a version with an accent, which is not a valid domain.
- Installments: with *n* installments, it is allowed if `price / n` reaches the tier minimum (R$ 50 up to 3x; R$ 80 from 4x to 6x; R$ 100 from 7x to 12x; the R$ 50 minimum also applies to 2x). E.g. 3x from R$ 150, 12x from R$ 1,200.
- PIX: 5% off the list price, **only on a product without a promotion** (promotions are not cumulative); the tool receives the product id and knows by itself whether there is a promotion. Combining payment methods is allowed **from R$ 2,000, inclusive** (the manual says "above"; same criterion as shipping).
- Shipping: only the city of Campo Grande counts as the "metropolitan region"; an order of exactly R$ 500 ships free (the manual says "above" and "below"); outside it, shipping depends on the ZIP code and the agent only gives the deadlines.
- Holidays: I consider national fixed-date holidays; movable ones (Carnival, Good Friday, Corpus Christi) and local ones are not included, and the agent tells the customer to confirm.
- After hours: the notice with the reopening time comes **before** the reply and **only once per session** (on the first reply in which the store is closed); if the store was open and closed in the middle of the conversation, it appears on the next reply.

Orders and returns

- The recorded `total_brl` is what counts, never the sum of the items at today's prices (on orders 3 and 20 they differ).
- Orders are treated as **online purchases** (they have tracking/delivery), so the 7-day right of withdrawal applies.
- There is no actual delivery date in the data: I use `estimated_delivery` as the receipt date. A shipped order has not started its window; pending/confirmed has nothing to return; cancelled neither.
- Withdrawal and preference exchange: 7 days; **defect: 30 days counted from receipt** (the manual says "after the purchase"; I chose receipt for consistency with the legal warranty and so shipping does not eat the window). After that, the 90-day legal warranty (from receipt) and the manufacturer's, with the store able to mediate.
- Personalized or "final sale" items cannot be identified in the data: the agent sends the customer to confirm with the staff.
- "Today" is the system's real date, overridable with `AGENT_TODAY`.
- Order verification is remembered in the session for 10 min of inactivity, capped at 30 min, scoped to the phone (all orders of that customer); no listing of orders by phone alone.

Complaints and hand-offs

- Saved locally with a sequential protocol, kind (`complaint` / `handoff`), description, contact and time; the contact is mandatory (without it nothing is saved and the agent asks). The "reply within 24 business hours" comes from the manual, but **no mechanism enforces it**: it is only the record.
- If the write fails, the tool tells the agent so, and the agent must not claim it registered.

Known limitations

- **The LLM can still embellish the text around the facts.** R$ values and dates are checked, the rest is not. In example `04` Maestro still says the staff "will generate the postage label" and that the refund counts "after the store receives the product", which the manual does not say. (Before the checker and the prompt rules it even invented partner stores and offered reservations.)
- **The checker is limited.** It only covers R$ values and dates; not percentages, number of installments or deadlines in days. It also rejects correct calculations done by the model (a deliberate decision). **I did not measure how many replies get rewritten** nor the false-positive rate: in the five example scenarios none fell into the safe message, but the sample is small.
- **No automated evaluation of the LLM's behaviour.** The tests cover rules, the loop and the checker, not the quality of the answers; the examples are a manual sample from one run.
- **Order verification is still weak for production**: phone + order number, no attempt limit (someone could try combinations) and no specific defence against prompt injection.
- **Complaints in a local file**: no queue, no notification to the staff, no case owner and no real deadline; the file holds phone/e-mail in plain text, with no encryption or retention policy (it is outside git, but it is personal data).
- **No business actions**: the agent only informs. It does not reserve, create or cancel orders; it records and forwards.
- **Simple search**: no typo tolerance, synonyms or use-based questions ("a good guitar for beginners").
- **History in memory only**; restarting the CLI resets the conversation and the verification.
- **Dependence on the free provider**: rate limits, overload (503) and retired models. The CLI shows a friendly message when the model fails, but it does not switch providers by itself.
- **Policy copied by hand from the PDF**, and **Portuguese only** for the agent's replies.
- Movable/local holidays and shipping outside Campo Grande are not computed; "store open now" and the after-hours notice always use the real clock (they ignore `AGENT_TODAY`).
- The examples show catalog, store/hours, out of stock + installments, return, and out of scope/privacy; the verification memory, complaints, the after-hours notice and the payment options are covered by tests and manual runs, but have no recorded example (the brief asks for 3 to 5 conversations).

What I would do with more time: an evaluation set (scenarios + LLM-as-judge, measuring rewrites and checker false positives); extend the checker to percentages, installments and deadlines; attempt limits and an audit log on order verification; notify the staff on complaints and store them with a retention policy; session persistence and a real channel (WhatsApp); search with typo tolerance and, if the manual grows, embeddings; a real human-agent flow for cancellations.

Use of AI coding assistants

I used **only Claude Code** (Anthropic's CLI, Sonnet model), with the `mattpocock-skills` plugin of *skills* (`to-spec`, `to-tickets`, `tdd`) to structure the work. The workflow, in practice:

1. **Specification**: Claude read the challenge PDF, the CSVs and the manual and wrote `SPEC.md` (problem, user stories, decisions, tests, out of scope), published as issue #1.

2. **Tickets**: the specification became 7 GitHub issues, as vertical slices with explicit dependencies (skeleton with persona → catalog → policies → orders → returns → examples → README).
3. **TDD per ticket**: before any test, Claude proposed the **test boundary** and I confirmed it. Then, one cycle at a time: failing test → minimal code → next test. Whenever there was a product decision, it asked me instead of choosing by itself.
4. **Real verification**: I ran the CLI with the real model and pasted the output; Claude adjusted from there.
5. **Second round (hardening)**: I asked Claude which of the LLM's tasks could become code. It mapped ten, with value and effort, and I agreed with the proposed order; they became six more issues (#9 to #14), following the same flow.
6. **Language**: I initially wrote the README in Portuguese (my choice at the time). When I found out that the deliverables must be in English, I asked Claude to translate the README and the documentation and generate a PDF; the agent itself keeps speaking Portuguese (see "Language").

What I decided: free provider and model (Gemini 3.7 Flash pinned; later I asked for Groq support), the use of LangChain, replies in pt-BR with code in English, `uv`, the "Maestro" persona and the opening introduction (my idea), the maximum number of calls per message and the fallback text, search rules (all words, searched fields, limit of 10, order by relevance/discount) and dropping the sort by location, status beating stock, alternatives ±30%, behaviour for a search with no result/no criteria, contacts and e-mail content, the online-orders assumption, the tiered installment rule, holiday handling, order verification by number + phone and no listing, the generic failure message, use of `estimated_delivery`, `AGENT_TODAY`, the defect window counted from receipt, and the example scenarios. In the second round: the after-hours notice only on the first reply of the session; complaints in a local file with only description and contact; the payment tool by product id; **session memory of the verification with a time limit** (the idea of the limit was mine; the 10 min inactivity, 30 min cap and phone scope I chose among the proposed options); and the checker's rules (evidence = tools + customer, derived values rejected, only R$ and dates, a safe message without values). And, at the end, the scope of the English translation and what goes into the PDF.

What Claude proposed and implemented: the tool architecture, the modules and tests, the conversion of the manual into sections, the script that records the examples, the LLM→code task mapping, the PDF build script, the translation, and the text of this README, always presenting options for me to choose from.

Assistant mistakes along the way (and how they were caught):

- It suggested `gemini-2.5-flash` from memory, already unavailable, and a Groq model that did not exist for my key: I caught it on the real run; the key's model list was queried and the default updated.
- A commit with "Closes #2" closed the issue on push, before the manual verification; it was reopened.
- A minimum-price test started with the wrong expected set (product 93 was missing); checking against the CSVs fixed it.
- A method ended up outside the class because of an editing error; the tests failed and revealed it.
- The initial ordering put discount before availability, against what we agreed; fixed.
- Maestro invented partner stores ("Music Center", "Lojinha do Som") and offered to "reserve the model", and the tools did not expose product descriptions/specifications: these showed up in my runs and when reading the recorded examples; they became prompt rules and a (tested) fix in the tool.
- It declared `langgraph` as a dependency without using it; removed while reviewing the README.
- A history test passed on the first run (the previous cycle's code already covered it); it was kept as protection.
- The PIX price of the Giannini came out `R$ 759,90` instead of `759,91` because of *float*; the assistant noticed it while checking the numbers in the tool output and fixed it with `Decimal` (test first).
- While implementing the checker, an edit duplicated methods of `Agent`: the central test kept failing and reading the file showed the duplication. A second edit failed because a pattern did not match literal characters in the file.
- **The checker crashed with the real model**: the regex treated the final dot of "…up to R$ 2.000." as part of the number and raised `InvalidOperation`. The unit tests did not catch it; re-recording the examples did. It was fixed with a stricter regex, a parser that never breaks the conversation, and tests for punctuation and unreadable numbers.
- The README said order verification was "stateless" after it stopped being so; fixed on the spot and reviewed here.
- Several new checker tests passed on the first run (the first cycle's code already covered them); so as not to trust a test that never fails, the assistant disabled the checker and confirmed that 5 of them break.

Claude Code skills used (`mattpocock-skills`)

The project was run with Claude Code and the `mattpocock-skills` plugin of *skills* (marketplace `claude-plugins-official`, version 1.3.1). A skill is a package of instructions that Claude Code loads for a kind of task; these ones push towards a disciplined engineering flow: **specification → tickets as vertical slices → TDD at test boundaries agreed beforehand**. I used four:

Skill	What it was for here	Where it shows in the repository
`setup-matt-`	Configure where issues live (GitHub, via `gh`), the triage-label vocabulary and the domain-documentation layout	`CLAUDE.md` ("Agent skills" block) and `docs/agents/` (`issue-tracker.md`, `triage-`

`pocock-skills`		`labels.md`, `domain.md`)
`to-spec`	Turn the challenge PDF, the CSVs and the manual into a specification (problem, user stories, decisions, tests, out of scope) and publish it as an issue labelled `ready-for-agent`	`SPEC.md` and issue #1
`to-tickets`	Break the specification into *tracer bullets*: vertical slices, each demonstrable on its own, with the blocking relations made explicit	Issues #2 to #8 (first round) and #9 to #14 (hardening), all with "Parent" and "Blocked by"
`tdd`	Drive each ticket in *red → green*, one test at a time, only at confirmed boundaries (*seams*)	The 163 tests in `tests/` and the commit history

How the skills' rules changed the work, in practice:

- **Test boundaries before any test.** The `tdd` skill does not allow writing a test at an unconfirmed boundary. So every ticket began with Claude proposing the boundary (what it catches and what it misses) and me confirming it: `Agent.respond` with a fake model; the `Catalog`, `Policies`, `Orders`, `assess_return` and `Complaints` interfaces. Everything outside that (CLI, thin tool layer, real LLM behaviour) was deliberately left without automated tests.
- **Vertical slices, not layers.** The tickets cut the whole path (data → rule → tool → prompt → CLI), so each could be verified with the real model before moving on (and was: I ran the CLI and pasted the output).
- **One cycle at a time, no "imagined" tests.** Tests were born from observable behaviour, with independent expected values (literals checked in the CSVs and the manual), avoiding tests that repeat the code's formula.
- **Product decisions become questions.** The `to-spec` skill says to synthesize the specification without interviewing the user (and to confirm only the test boundaries). Since I asked to take part in the decisions, Claude also asked me every product decision (search rules, installments, return window, verification-memory time...), and the answers were recorded in the issues.
- **Labels and traceability.** Issues with `ready-for-agent`, a status comment on each one, and closing only after verifying with the real model (a commit with "Closes #2" once closed an issue too early; it was reopened).

Where I departed from the skills, or where they hit limits:

- In some cycles I grouped related tests before implementing (for example the format and expiry ones for order verification) instead of one test at a time; and several tests passed on the first run because the previous cycle's code already covered them. When that happened I checked that they really fail (mutation check on the reply checker).
- The installed `gh` (2.4) is old: it has no native sub-issues, no issue dependencies and no `--comment` on `close`. Dependencies were left as text ("Blocked by") and closing was comment + `close`.
- The domain-modelling skill (`GLOSSARY.md` and ADRs) is configured in `docs/agents/domain.md`, but **I did not use it**: decisions were recorded in the issues and in the README. I also did not use the code-review, `grilling`, `prototype` or `wizard` skills; the brief asked for a prototype and the specification → tickets → TDD cycle was enough.
- `CLAUDE.md` and `docs/agents/` configure the development tool, not the Maestro agent: the agent itself depends neither on any skill nor on Claude Code.

Repository structure

```
src/emporio_agent/   agent code
tests/               163 tests (fake model + real CSVs)
data/                store CSVs and PDF; data/policies/ = the manual as .md sections (Portuguese)
examples/            5 conversations recorded with the real model (English headers, Portuguese transcripts)
scripts/             example recording and documentation-PDF build
docs/                tool configuration (docs/agents/) and maestro-documentation.pdf
SPEC.md              initial specification
complaints.jsonl     (created at run time, ignored by git) complaints and hand-offs
```

2. Specification (SPEC.md)

Source: `SPEC.md`

> **Status:** initial specification, written at the start of the project and kept as such. Decisions made later (provider, verification memory, the hardening round with issues #9–#14, the English-only documentation) are recorded in the README and in the issues.

Problem Statement

The customer-service team at Empório da Música (a musical-instrument store in Campo Grande, MS, Brazil) is overloaded with repetitive questions: opening hours, address, product price and availability, order status, exchange/shipping/payment rules. Customers expect a fast reply (within 5 minutes) in an informal but professional tone, and the store must not give wrong information about price, stock or deadlines. In addition, the Artefact technical challenge requires a Python prototype, a public repository with a real commit history, a README justifying the technical decisions, and 3–5 example conversations.

Solution

A text-message customer-service agent, in Python, with the store's persona (a friend who knows music, informal but professional). The agent uses function calling over the store's data (CSVs, accessed through live lookup tools) and over the policies (the PDF manual, converted into searchable sections). It decides by itself when to look up data, when to look up policies, and when to politely decline (out of scope, e.g. accessories, which the store does not sell). Interface: a simple chat CLI, with per-session history. The agent talks to customers in Brazilian Portuguese; everything else in the repository is in English.

User Stories

1. As a customer, I want to ask "Quais violões custam até R$ 1000?" ("Which acoustic guitars cost up to R$ 1000?"), so that I can see options filtered by category and price from the real catalog.
2. As a customer, I want to ask "Quanto custa o Takamine GD20?", so that I get the current price even if I write the name partially or with different casing/accents.
3. As a customer, I want to see the promotional price together with the original price and discount percent, so that I understand the offer transparently.
4. As a customer, I want the agent to never promise a discount from an expired promotion (is_active=0), so that I am not misled.
5. As a customer, I want to know if a product is in stock, so that I don't go to the store or order something unavailable.
6. As a customer, I want alternatives suggested when a product is out of stock (stock 0), so that I can still buy something similar.
7. As a customer, I want to be told when a product is discontinued and be offered equivalent models, so that I'm not left at a dead end.
8. As a customer, I want to be told when a product is "coming soon" (not yet purchasable), so that I'm not told it's available.
9. As a customer, I want to ask for a product by category (guitarras, baixos, ukuleles, etc.) and by specs (cordas nylon, cor), so that I can browse.
10. As a customer, I want to ask about the status of my order, so that I know if it's pending, confirmed, shipped, delivered or cancelled.
11. As a customer, I want to receive my tracking code and estimated delivery when the order is shipped, so that I can follow it.
12. As a customer, I want to be told the reason a cancelled order was cancelled, so that I understand what happened.
13. As a customer, I want the agent to verify that an order belongs to me before revealing details (e.g. matching name/phone/email), so that my data stays private (LGPD).
14. As a customer, I want to ask "Me arrependi da compra, posso devolver?" ("I regret the purchase, can I return it?"), so that the agent applies the 7-day right-of-withdrawal rule against my order's delivery date.
15. As a customer, I want the agent to tell me if my order is outside the return window and what alternatives exist (warranty, 30-day defect exchange), so that I know my rights.
16. As a customer, I want to know return shipping is paid by the store in arrependimento cases, so that I'm not surprised by cost.
17. As a customer, I want to know which items aren't eligible for exchange (personalizados, venda final, boquilhas), so that I'm not misinformed.

18. As a customer, I want to ask about defect exchange (30 days) and warranty (90 days legal + manufacturer), so that I know what applies.
19. As a customer, I want to know the store address, phone/WhatsApp and email, so that I can reach or visit the store.
20. As a customer, I want to know opening hours and whether the store is open right now, so that I can plan a visit.
21. As a customer, I want the agent outside business hours to tell me when the store returns, so that I know when to expect a human.
22. As a customer, I want to know accepted payment methods and installment rules (3x/6x/12x, minimum installment), so that I can plan the purchase.
23. As a customer, I want the agent to compute installments for a given product price and tell me the max number of installments, so that I see the real value per parcela.
24. As a customer, I want to know PIX gives 5% off but not on already-promotional prices, so that I understand non-cumulative rules.
25. As a customer, I want to know delivery fees and times (free above R$ 500 in Campo Grande, R$ 35 below; PAC/SEDEX/Jadlog for other cities), so that I can compare.
26. As a customer, I want to be told large instruments may require special freight quote, so that I'm not surprised.
27. As a customer, I want to be redirected politely when I ask for accessories (cordas, palhetas, cabos, pedais, amplificadores, cases), so that I know the store doesn't sell them.
28. As a customer, I want off-topic questions (politics, recipes, code) to be declined gracefully with a return to store topics, so that the agent stays on role.
29. As a customer, I want to make a complaint and be told it will be registered and answered within 24 business hours, so that I feel heard.
30. As a customer, I want questions about data privacy answered (use of data, deletion), so that I trust the store.
31. As a customer, I want the agent to remember what we talked about earlier in the conversation ("e esse tem em outra cor?"), so that I don't repeat myself.
32. As a customer, I want to be greeted by name when identified and asked how the agent can help, and the conversation to close cordially, so that service follows the store's standard flow.
33. As a customer, I want the agent to say it doesn't know (and suggest contacting the store) rather than invent an answer, so that I get no incorrect info.
34. As a store attendant, I want the agent to answer only with data from the system, so that price/stock/deadline never become false advertising.
35. As a store attendant, I want the agent to escalate doubts about policy application to the management/a human, so that edge cases aren't improvised.
36. As a gerente, I want the agent's tone aligned with the store's identity, so that the brand is consistent.
37. As an Artefact evaluator, I want a README with run instructions, technical justifications, known limitations, assumptions and AI-assistant workflow, so that I understand the reasoning.
38. As an avaliador, I want 3–5 example conversations in the repo (including at least one real-time data + policy case), so that I can see the agent working without running it.
39. As an avaliador, I want a commit history showing real progress, so that I can follow the work.
40. As a desenvolvedor, I want the model provider configured via environment variables, so that I can swap providers without code changes.
41. As a desenvolvedor, I want automated tests of the agent's tools, so that data rules are verified without calling an LLM.

Implementation Decisions

- **Approach:** function-calling agent (tool-use loop) rather than SQL agent or full RAG. Data is small (65 products, 19 orders, 49 customers), so deterministic, typed tools are safer and cheaper than LLM-generated SQL. Policies are short (~8 pages): split into sections and exposed through a policy-lookup tool (keyword/embedding-free retrieval over sections is sufficient; mention embeddings as future work).
- **Modules (deep, small interfaces):**
- *Data store:* loads and normalizes CSVs once (typed prices, parsed specs JSON, joins products↔categories↔promotions, orders↔items↔customers). The filenames in `data/` have a long prefix; the loader hides that.
- *Catalog tools:* search products (text query with accent/case-insensitive fuzzy match, category, max/min price, only-available filter), get product detail with effective price (active promotion applied; original price and percent shown), stock and status (active/discontinued/coming_soon), suggest alternatives (same category, similar price, in stock).
- *Order tools:* get order status by order id, only after customer verification (customer identified by matching phone/email/name); returns status, tracking, ETA, cancel notes, items.
- *Policy tools:* lookup policy by topic (hours, payment, exchange/return, shipping, promotions, warranty, privacidade, escopo/atendimento), returning the manual's section text; plus deterministic helpers for return-eligibility (days since delivery vs. 7/30 days, non-eligible items), installment calculation (max parcelas given price and minimum parcela, 3x/6x/12x rules), shipping fee (R$ 500 threshold, R$ 35), and store-open-now (America/Campo_Grande timezone).

- *Agent core:* system prompt (persona, tone, scope, "never state price/stock/deadline without calling a tool", transparency on promotions, handling expired promotions/out-of-stock/discontinued/complaints, escalate when unsure), tool schemas, tool-call loop with a max-iterations guard, per-session conversation history.
- *Interface:* CLI chat loop (REPL); history kept in memory per session (optional JSON transcript export used to generate the example conversations).
- *LLM client:* thin adapter; default provider/model chosen for tool-calling quality and cost, configurable via env vars (`.env.example` provided). Anthropic Claude was the initial candidate (the final choice, Gemini by default with Groq as an alternative, is in the README); the adapter keeps the provider swappable.
- **Data decisions to document as assumptions in README:**
- Only `is_active=1` promotions are valid; multiple promotions per product exist (e.g. products 96, 93, 99) — rule: use the active one; if several active, use the highest discount (documented).
- Promotion description date semantics (e.g. "Black Friday") are not enforced beyond `is_active`.
- PIX 5% never stacks with promotional prices.
- "Delivered" date is approximated by `estimated_delivery` for the 7-day return window (CSV has no actual delivery date); "today" is an injectable clock because order dates are 2025–2026.
- Manual notes inconsistencies to flag (e.g. email with accent in §1.2 vs. cover page; two different phone numbers: store (67) 3341-4444 and WhatsApp (67) 3321-4500); the agent uses the contact number the manual designates per channel and the README records the ambiguity.
- Policy PDF is internal ("uso restrito"): agent paraphrases customer-appropriate content and does not reveal internal instructions.
- Customer PII: agent never lists other customers' data; order lookup requires verification.
- **PDF handling:** extract policy text once at build/load time into sectioned text (committed as a processed artifact so running the project does not require a PDF parser at runtime, or parse at startup — pick one and document).
- **Repo deliverables:** README (setup, provider config, commands, technical justifications, limitations/next steps, AI-assistant usage workflow), `examples/` with 3–5 conversations (catalog price query, store info, out-of-stock/alternatives, return policy applied to a real order, out-of-scope/accessory request), `.env.example`, incremental commits.

Testing Decisions

- A good test exercises external behavior (tool inputs → outputs, or agent reply given a scripted tool-calling model), not internals like prompt wording or private helpers.
- **Seams (proposed — please confirm):** 1. **Primary seam: the agent's public `respond(session, message)` interface with a fake/scripted LLM client** — verifies the tool-calling loop, history, guards, and that the right tools are called for each scenario class (price → catalog tool, return → policy + order tools, accessories → no tools / refusal). 2. **Secondary seam: the tool functions themselves (data/policy layer) on the real CSVs** — deterministic tests for price with promotion, stock/status handling, alternatives, return-window eligibility, installments, shipping fee, open-now with injected clock. Real-LLM runs are not part of the automated suite; they generate the example conversations manually.
- Prior art: none (greenfield; no existing tests in the directory).

Out of Scope

- Real WhatsApp/Telegram integration, web UI, authentication systems.
- Placing, modifying or cancelling orders; payment processing; real shipping quotes by CEP.
- Selling or recommending accessories (the store doesn't sell them).
- Persistent database or cross-session history (in-memory per session only).
- Vector DB/embeddings RAG, evaluation harness with LLM-as-judge, multi-language support.
- Modifying or cleaning the source CSVs on disk (normalization happens at load time).

Further Notes

- This spec was later published as issue #1 on GitHub (label `ready-for-agent`); this file is the version as written at the start of the project.
- The challenge statement asks for assumptions to be documented in the README rather than blocking on questions; follow that for any ambiguity.
- Deadline/date of delivery is in the selection-process email (not in the PDF); don't push to the repo after it.
- Data quirks seen: 62 active / 2 discontinued / 1 coming_soon products, 2 with zero stock; order statuses pending/confirmed/shipped/delivered/cancelled; only shipped/delivered orders have tracking codes.

3. Development-tool configuration (docs/agents)

Source: `docs/agents/domain.md`

Domain Docs

How the engineering skills should consume this repo's domain documentation when exploring the codebase.

Before exploring, read these

- **`GLOSSARY.md`** at the repo root, or
- **`GLOSSARY-MAP.md`** at the repo root if it exists: it points at one `GLOSSARY.md` per context. Read each one relevant to the topic.
- **`docs/adr/`**: read ADRs that touch the area you're about to work in. In multi-context repos, also check `src/<context>/docs/adr/` for context-scoped decisions.

If any of these files don't exist, **proceed silently**. Don't flag their absence; don't suggest creating them upfront. The `/domain-modeling` skill (reached via `/grill-with-docs` and `/improve-codebase-architecture`) creates them lazily when terms or decisions actually get resolved.

File structure

Single-context repo (most repos):

```
/
├── GLOSSARY.md
├── docs/adr/
│   ├── 0001-event-sourced-orders.md
│   └── 0002-postgres-for-write-model.md
└── src/
```

Multi-context repo (presence of `GLOSSARY-MAP.md` at the root):

```
/
├── GLOSSARY-MAP.md
├── docs/adr/                          ← system-wide decisions
└── src/
    ├── ordering/
    │   ├── GLOSSARY.md
    │   └── docs/adr/                  ← context-specific decisions
    └── billing/
        ├── GLOSSARY.md
        └── docs/adr/
```

Use the glossary's vocabulary

When your output names a domain concept (in an issue title, a refactor proposal, a hypothesis, a test name), use the term as defined in `GLOSSARY.md`. Don't drift to synonyms the glossary explicitly avoids.

If the concept you need isn't in the glossary yet, that's a signal: either you're inventing language the project doesn't use (reconsider) or there's a real gap (note it for `/domain-modeling`).

Flag ADR conflicts

If your output contradicts an existing ADR, surface it explicitly rather than silently overriding:

> *Contradicts ADR-0007 (event-sourced orders), but worth reopening because...*

Source: `docs/agents/issue-tracker.md`

Issue tracker: GitHub

Issues and specs for this repo live as GitHub issues. Use the `gh` CLI for all operations.

Conventions

- **Create an issue**: `gh issue create --title "..." --body "..."`. Use a heredoc for multi-line bodies.
- **Read an issue**: `gh issue view <number> --json number,title,body,labels,comments`.
- **List issues**: `gh issue list --state open --json number,title,body,labels,comments --jq '[.[] | {number, title, body, labels: [.labels[].name], comments: [.comments[].body]}]'` with appropriate `--label` and `--state` filters.
- **Make an issue a sub-issue of a parent**: `gh issue create --parent <parent> ...`, or `gh issue edit <parent> --add-sub-issue <child>` afterwards (`gh` 2.94+). Older `gh`: `gh api --method POST repos/<owner>/<repo>/issues/<parent>/sub_issues -F sub_issue_id=<child-db-id>` (database id, as in **Blocking** below). Without sub-issues, put `Part of #<parent>` at the top of the child body.
- **Comment on an issue**: `gh issue comment <number> --body "..."`
- **Apply / remove labels**: `gh issue edit <number> --add-label "..."` / `--remove-label "..."`
- **Close**: `gh issue close <number> --comment "..."`

Infer the repo from `git remote -v`; `gh` does this automatically when run inside a clone.

Pull requests as a triage surface

PRs as a request surface: no. *(Set to `yes` if this repo treats external PRs as feature requests; `/triage` reads this flag.)*

When set to `yes`, PRs run through the same labels and states as issues, using the `gh pr` equivalents:

- **Read a PR**: `gh pr view <number> --comments` and `gh pr diff <number>` for the diff.
- **List external PRs for triage**: `gh api --paginate 'repos/{owner}/{repo}/pulls?state=open' --jq '.[] | select(.author_association | IN("OWNER","MEMBER","COLLABORATOR") | not) | {number, title, author: .user.login, author_association, labels: [.labels[].name]}'`.
- **Comment / label / close**: `gh pr comment`, `gh pr edit --add-label`/`--remove-label`, `gh pr close`.

GitHub shares one number space across issues and PRs, so a bare `#42` may be either: resolve with `gh pr view 42` and fall back to `gh issue view 42`.

When a skill says "publish to the issue tracker"

Create a GitHub issue.

When a skill says "fetch the relevant ticket"

Read it as in **Read an issue** above.

Wayfinding operations

Used by `/wayfinder`. The **map** is a single issue with **child** issues as tickets.

- **Map**: a single issue labelled `wayfinder:map`, holding the Notes / Decisions-so-far / Fog body. `gh issue create --label wayfinder:map`.
- **Child ticket**: an issue linked to the map as a GitHub sub-issue (see **Make an issue a sub-issue of a parent**). Where sub-issues aren't enabled, add the child to a task list in the map body and put `Part of #<map>` at the top of the child body. Labels: `wayfinder:<type>` (`research`/`prototype`/`grilling`/`task`). Once claimed, the ticket is assigned to the driving dev.
- **Blocking**: GitHub's **native issue dependencies**, the canonical, UI-visible representation. Add an edge with `gh api --method POST repos/<owner>/<repo>/issues/<child>/dependencies/blocked_by -F issue_id=<blocker-db-id>`, where `<blocker-db-id>` is the blocker's numeric **database id** (`gh api repos/<owner>/<repo>/issues/<n> --jq .id`, *not* the `#number` or `node_id`). GitHub reports `issue_dependencies_summary.blocked_by` (open blockers only, the live gate). Where dependencies aren't available, fall back to a `Blocked by: #<n>, #<n>` line at the top of the child body. A ticket is unblocked when every blocker is closed.

- **Frontier query**: list the map's open children (`gh issue list --state open`, scoped to the map's sub-issues / task list), drop any with an open blocker (`issue_dependencies_summary.blocked_by > 0`, or an open issue in the `Blocked by` line) or an assignee; first in map order wins.
- **Claim**: `gh issue edit <n> --add-assignee @me`, the session's first write.
- **Resolve**: `gh issue comment <n> --body "<answer>"`, then `gh issue close <n>`, then append a context pointer (gist + link) to the map's Decisions-so-far.

Source: `docs/agents/triage-labels.md`

Triage Labels

The skills speak in terms of five canonical triage roles. This file maps those roles to the actual label strings used in this repo's issue tracker.

Label in mattpocock/skills	Label in our tracker	Meaning
`needs-triage`	`needs-triage`	Maintainer needs to evaluate this issue
`needs-info`	`needs-info`	Waiting on reporter for more information
`ready-for-agent`	`ready-for-agent`	Fully specified, ready for an AFK agent
`ready-for-human`	`ready-for-human`	Requires human implementation
`wontfix`	`wontfix`	Will not be actioned

When a skill mentions a role (e.g. "apply the AFK-ready triage label"), use the corresponding label string from this table.

Edit the "Label in our tracker" column to match whatever vocabulary you actually use.

4. Appendix: example conversations (Portuguese transcripts)

Source: `examples/01-catalog.md`

Catalog search and product details

Category and price filter over the real catalog; a follow-up question about one model.

- Provider/model: `groq` / `openai/gpt-oss-120b`
- Recorded on: 2026-11-03 13:37
- The conversation is in Portuguese on purpose: the agent answers Brazilian customers in pt-BR.
- The "" blocks show the tools the agent called (arguments and result).

Maestro: Oi! Eu sou o Maestro, assistente virtual da Empório da Música.

Posso te ajudar com: • Preços, estoque e promoções dos instrumentos • Status do seu pedido (preciso do número do pedido e do telefone usado na compra) • Horários, endereço, formas de pagamento e parcelamento • Frete e prazos de entrega • Trocas, devoluções e garantia

Como posso ajudar?

Customer: Quais opções de violões disponíveis custando até R$1000?

> **Tool call:** `search_products(category='Violões', max_price=1000, query='')`
>
> ```
> [id 83] Tagima Memphis AC-39 Nylon Natural (Violões): R$ 429,90 — disponível
> [id 85] Rozini RC-104 Nylon Estudante (Violões): R$ 459,90 — disponível
> [id 89] Shelby SN-100 Folk Aço Preto (Violões): R$ 549,00 — disponível
> [id 88] Giannini GF-1R Aço Folk Sunburst (Violões): R$ 579,90 — disponível
> [id 81] Yamaha C40 Nylon Natural (Violões): R$ 599,90 — disponível
> [id 84] Giannini GN-15 Nylon Cedr Natural (Violões): R$ 689,00 — disponível
> [id 87] Yamaha F310 Aço Natural (Violões): R$ 699,90 — disponível
> [id 86] Tagima Dallas Tuner Aço Natural (Violões): R$ 749,00 — disponível
> [id 107] Shelby SGD-19 […]
> ```

Maestro: Aqui vão os violões que temos no catálogo com preço até R$ 1.000:

Código	Modelo	Preço
83	Tagima Memphis AC-39 Nylon – Natural	R$ 429,90
85	Rozini RC-104 Nylon – Estudante	R$ 459,90
89	Shelby SN-100 Folk Aço – Preto	R$ 549,00
88	Giannini GF-1R Aço Folk – Sunburst	R$ 579,90
81	Yamaha C40 Nylon – Natural	R$ 599,90
84	Giannini GN-15 Nylon – Cedro Natural	R$ 689,00
87	Yamaha F310 Aço – Natural	R$ 699,90
86	Tagima Dallas Tuner Aço – Natural	R$ 749,00
107	Shelby SGD-195E Elétrico Aço – Sunburst	R$ 799,00
82	Yamaha C70 Nylon – Natural	R$ 849,00

Se quiser saber mais detalhes de algum modelo (especificações, disponibilidade em cores, etc.) ou comparar opções, é só me dizer o código do produto que eu trago as informações completas. Precisa de ajuda com pagamento, frete ou alguma outra coisa? Estou à disposição!

Customer: Me fala mais do Yamaha F310.

> **Tool call:** `get_product_details(product_id=87)`
>
> ```
> [id 87] Yamaha F310 Aço Natural (Violões): R$ 699,90 — disponível
> Descrição: Violão folk aço Yamaha com corpo robusto e timbre encorpado. Um dos modelos mais vendidos do mundo para iniciantes e intermediários.
> Especificações: top: Spruce, back_sides: Meranti, neck: Nato, strings: steel, scale: 634mm, electronics: no, color: Natural
> ```

Maestro: Yamaha F310 – Aço (Natural)

- **Preço:** R$ 699,90 (disponível)
- **Descrição:** Violão folk de aço da Yamaha, muito popular entre iniciantes e músicos intermediários. O corpo robusto oferece um timbre encorpado e equilibrado, ideal para diversos estilos.
- **Especificações técnicas:**
- **Tampo (top):** Spruce (abeto)
- **Laterais e fundo (back & sides):** Meranti
- **Braço (neck):** Nato
- **Cordas:** Steel (aço)
- **Escala:** 634 mm
- **Eletrônica:** Não possui (acústico puro)
- **Cor:** Natural

Se quiser saber sobre formas de pagamento, parcelamento ou frete para a sua cidade, é só me dizer. Posso também comparar com outros modelos ou ajudar com qualquer outra dúvida que você tenha!

Source: `examples/02-store-info-and-hours.md`

Store information and whether it is open now

Manual data (address) and a live check of the opening hours.

- Provider/model: `groq` / `openai/gpt-oss-120b`
- Recorded on: 2026-11-03 13:37
- The conversation is in Portuguese on purpose: the agent answers Brazilian customers in pt-BR.
- The "" blocks show the tools the agent called (arguments and result).

Maestro: Oi! Eu sou o Maestro, assistente virtual da Empório da Música.

Posso te ajudar com: • Preços, estoque e promoções dos instrumentos • Status do seu pedido (preciso do número do pedido e do telefone usado na compra) • Horários, endereço, formas de pagamento e parcelamento • Frete e prazos de entrega • Trocas, devoluções e garantia

Como posso ajudar?

Customer: Qual o endereço da loja?

> **Tool call:** `get_policy(topic='loja')`
>
> ```
> # Sobre a Empório da Música
>
> Loja especializada em instrumentos musicais, fundada em 2008 em Campo Grande, Mato Grosso do Sul. Mais de 15 anos de atuação, referência regional na venda de instrumentos para músicos iniciantes, intermediários e profissionais. Slogan: "Sua música começa aqui."
>
> Catálogo: mais de 300 instrumentos de marcas nacionais e internacionais (guitarras, baixos, violões, baterias, teclados, instrumentos de sopro, cordas orquestrais e ukuleles). Trabalhamos exclusivamente com instrumentos musicais: não comercializamos acessórios como cordas, palhetas, cabos, cases, pedais ou  […]
> ```

Maestro: A gente fica na Rua **14 de Maio, 3200**, no Centro de Campo Grande – MS, CEP 79202-333. Se precisar de mais alguma coisa, é só chamar!

Customer: E vocês estão abertos agora?

> **Tool call:** `is_store_open_now()`
>
> ```
> A loja está aberta agora.
> ```

Maestro: Sim, a loja está aberta agora. Precisa de alguma informação ou ajuda com algum instrumento?

Source: `examples/03-out-of-stock-and-installments.md`

Out-of-stock product, alternatives and installments

Active product with zero stock: the agent does not confirm availability, suggests alternatives and computes installments.

- Provider/model: `groq` / `openai/gpt-oss-120b`
- Recorded on: 2026-11-03 13:38
- The conversation is in Portuguese on purpose: the agent answers Brazilian customers in pt-BR.
- The "" blocks show the tools the agent called (arguments and result).

Maestro: Oi! Eu sou o Maestro, assistente virtual da Empório da Música.

Posso te ajudar com: • Preços, estoque e promoções dos instrumentos • Status do seu pedido (preciso do número do pedido e do telefone usado na compra) • Horários, endereço, formas de pagamento e parcelamento • Frete e prazos de entrega • Trocas, devoluções e garantia

Como posso ajudar?

Customer: Vocês têm o violão Giannini GF-3D?

> **Tool call:** `search_products(category='Violões', query='Giannini GF-3D')`
>
> ```
> [id 96] Giannini GF-3D Dreadnought Sunburst (Violões): R$ 799,90 — sem estoque no momento
> ```

Maestro: Oi! Temos o violão Giannini GF-3D Dreadnought Sunburst, mas ele está sem estoque no momento. Se quiser, posso buscar algum modelo similar que esteja disponível ou deixar seu contato para avisarmos assim que chegar. Me avisa o que prefere!

Customer: Quanto fica o Shelby SGD-195E? Dá pra parcelar em quantas vezes?

> **Tool call:** `get_product_details(product_id=0)`
>
> ```
> Produto não encontrado.
> ```

> **Tool call:** `search_products(query='Shelby SGD-195E')`
>
> ```
> [id 107] Shelby SGD-195E Elétrico Aço Sunburst (Violões): R$ 799,00 — disponível
> ```

> **Tool call:** `get_product_details(product_id=107)`
>
> ```
> [id 107] Shelby SGD-195E Elétrico Aço Sunburst (Violões): R$ 799,00 — disponível
> Descrição: Violão eletroacústico dreadnought com captação e afinador integrado. Boa opção de custo-benefício para quem precisa de amplificação.
> Especificações: top: Spruce, back_sides: Sapele, neck: Nato, strings: steel, scale: 650mm, electronics: yes, color: Sunburst
> ```

> **Tool call:** `get_installments(price=799)`
>
> ```
> Até 7x de R$ 114,14 sem juros. Opções: 1x de R$ 799,00, 2x de R$ 399,50, 3x de R$ 266,33, 4x de R$ 199,75, 5x de R$ 159,80, 6x de R$ 133,17, 7x de R$ 114,14
> ```

Maestro: O Shelby SGD-195E está por **R$ 799,00** e está disponível em estoque.

Parcelamento: até 7x sem juros de R$ 114,14. Também dá pra pagar à vista (1x R$ 799,00) ou em 2 a 6 vezes, conforme a tabela abaixo:

- 2x de R$ 399,50
- 3x de R$ 266,33
- 4x de R$ 199,75
- 5x de R$ 159,80
- 6x de R$ 133,17

Quer saber mais sobre o produto, frete para sua cidade ou como finalizar a compra? É só me dizer!

Source: `examples/04-real-order-return.md`

Return of a real order (data + policy)

Verification by order number and phone, the return reason and the 7-day window applied to order 7.

- Provider/model: `groq` / `openai/gpt-oss-120b`
- Recorded on: 2026-11-03 13:39 (date simulated by the agent: `AGENT_TODAY=2026-02-20`)
- The conversation is in Portuguese on purpose: the agent answers Brazilian customers in pt-BR.
- The "" blocks show the tools the agent called (arguments and result).

Maestro: Oi! Eu sou o Maestro, assistente virtual da Empório da Música.

Posso te ajudar com: • Preços, estoque e promoções dos instrumentos • Status do seu pedido (preciso do número do pedido e do telefone usado na compra) • Horários, endereço, formas de pagamento e parcelamento • Frete e prazos de entrega • Trocas, devoluções e garantia

Como posso ajudar?

Customer: Me arrependi da minha compra, posso devolver meu pedido?

Maestro: Oi! Claro, vamos verificar a possibilidade de devolução.

Para isso, preciso do número do seu pedido e do telefone que foi usado na compra. Você pode me passar esses dados?

Customer: Pedido 7, telefone (67) 99789-0123.

> **Tool call:** `check_return_eligibility(order_id=7, phone='(67) 99789-0123', reason='arrependimento')`
>
> ```
> Dentro do prazo: o cliente pode solicitar até 24/02/2026 (restam 4 dia(s)).
>
> Assuma compra online. Se o item for personalizado/sob encomenda ou venda final, ou a compra tiver sido na loja física,
> confirme com a equipe.
>
> Regras do manual:
> # Política de trocas e devoluções
>
> A Empório da Música segue o Código de Defesa do Consumidor (Lei nº 8.078/90).
>
> ## Direito de arrependimento (compras online)
> - O cliente pode solicitar a devolução em até 7 (sete) dias corridos após o recebimento do produto, sem necessidade de
> justificativa.
> - O produto deve estar na embalagem original, sem sinais de uso, com […]
> ```

Maestro: Ótimo, seu pedido está dentro do prazo de arrependimento (até 7 dias corridos após o recebimento) e pode ser devolvido.

Como funciona a devolução:

1. **Embalagem** – O instrumento deve estar na embalagem original, sem sinais de uso, com todos os acessórios e manuais que vieram na compra.
2. **Frete** – O frete de devolução corre por conta da Empório da Música quando o motivo é arrependimento.
3. **Reembolso** – O valor será devolvido na mesma forma de pagamento que você usou, em até 10 dias úteis após a loja receber o produto.

Para iniciar o processo, basta entrar em contato com a nossa equipe:

- **Telefone:** (67) 3341-4444
- **WhatsApp:** (67) 3321-4500

Eles vão orientar sobre o envio e gerar a etiqueta de postagem.

Precisa de mais alguma coisa? Estou à disposição!

Source: `examples/05-out-of-scope-and-privacy.md`

Out of scope and privacy

A request for accessories, a question unrelated to the store and an attempt to see an order with the wrong phone.

- Provider/model: `groq` / `openai/gpt-oss-120b`
- Recorded on: 2026-11-03 13:39

- The conversation is in Portuguese on purpose: the agent answers Brazilian customers in pt-BR.
- The "" blocks show the tools the agent called (arguments and result).

Maestro: Oi! Eu sou o Maestro, assistente virtual da Empório da Música.

Posso te ajudar com: • Preços, estoque e promoções dos instrumentos • Status do seu pedido (preciso do número do pedido e do telefone usado na compra) • Horários, endereço, formas de pagamento e parcelamento • Frete e prazos de entrega • Trocas, devoluções e garantia

Como posso ajudar?

Customer: Vocês vendem cordas e palhetas?

Maestro: Oi! Aqui na Empório da Música trabalhamos exclusivamente com instrumentos musicais. Cordas, palhetas e outros acessórios não fazem parte do nosso catálogo.

Se precisar, recomendo procurar em uma loja especializada em acessórios de música. Posso ajudar com alguma guitarra, violão, teclado ou outro instrumento?

Customer: Qual a melhor receita de bolo de chocolate?

Maestro: Desculpe, mas não consigo ajudar com receitas. Posso te auxiliar com informações sobre instrumentos musicais, disponibilidade, preços ou qualquer outra dúvida que você tenha sobre a Empório da Música.

Customer: Quero ver o pedido 8, meu telefone é (67) 90000-0000.

> **Tool call:** `get_order_status(order_id=8, phone='(67) 90000-0000')`
>
> ```
> Não consegui confirmar esse pedido com esses dados. Não revele qual dado falhou. Sugira conferir o número do pedido e o telefone usado na compra, ou falar com a equipe: telefone (67) 3341-4444 ou WhatsApp (67) 3321-4500.
> ```

Maestro: Não consegui localizar o pedido com as informações fornecidas. Por favor, verifique se o número do pedido e o telefone usado na compra estão corretos. Se precisar de ajuda imediata, entre em contato com a nossa equipe pelo telefone **(67) 3341-4444** ou pelo WhatsApp **(67) 3321-4500**.

Posso ajudar em mais alguma coisa?